

Sequence Diagrams

Introduction

Sequence Diagram

A **Sequence Diagram** is a behavioral diagram in Unified Modeling Language (UML) used to represent interactions between objects in a time-ordered sequence. It focuses on the order and timing of messages exchanged between system elements.

Key Characteristics

- Shows interactions between objects over time
- Represents message flow and execution order
- Useful for understanding system behavior and workflows

Why Sequence Diagrams Are Used

- **Dynamic Behavior Visualization** — Clearly shows runtime interactions
- **Clear Communication** — Easy to understand system logic without code
- **Use-Case Analysis** — Explains how use cases are executed
- **System Design** — Helps define communication between components
- **Documentation** — Serves as long-term system reference
- **Debugging** — Identifies bottlenecks and logical errors

Sequence Diagram Syntax

Basic Syntax Elements

A sequence diagram is constructed using the following core syntactic elements:

- An Actor
- An Object
- A Lifeline

- A Message
- Object Destruction

Note

The syntax focuses on **who interacts with whom** and **in what order over time**.

Major Components

A sequence diagram consists of two major components:

- **Active Objects**
- **Communications (Messages) between Active Objects**

Messages are exchanged between active objects to describe system behavior.

Active Objects

Definition

Active Object

Active objects are any objects or actors that participate in a system by sending and/or receiving messages.

Characteristics of Active Objects

- Play a role in system behavior
- Participate through message exchange
- Placed horizontally across the top of the diagram

Types of Active Objects

- **Actors** (external to the system)
- **Objects or Classes** (internal to the system)

Object Representation

Object Definition

Object

An object represents an instance or class that exists within the system and participates in interactions.

Object Naming Syntax

[instanceName] [:className]

Valid Object Naming Forms

- **Class name only:** :ClassName
- **Instance name only:** objectName
- **Instance and class name:** objectName:ClassName

Examples

- myBirthday
- :Date

Actors

Actor

An actor is a person or external system that interacts with the system and derives benefit from it.

Characteristics of Actors

- External to the system boundary
- Participate by sending and/or receiving messages
- Often identified from use case diagrams

Communications Between Active Objects

Messages

Message

A message represents communication between two active objects in a sequence diagram.

Messages are used when an object needs:

- To activate a process of another object
- To pass information to another object

Message Representation

- Represented by an arrow between lifelines
- Labeled with at least the message name

- Arguments and control information may be included
- Self-messages are allowed

Message Syntax

`aMessage()`

Types of Messages

Synchronous Messages

- Caller waits until execution completes
- Control flow is interrupted
- Common in method calls

Asynchronous Messages

- Caller does not wait for response
- Execution continues immediately
- Used for notifications and progress updates

Return Messages

- Represent return of control to caller
- Shown using dashed arrows
- Return values modeled only when necessary

Sequence Diagram Notations

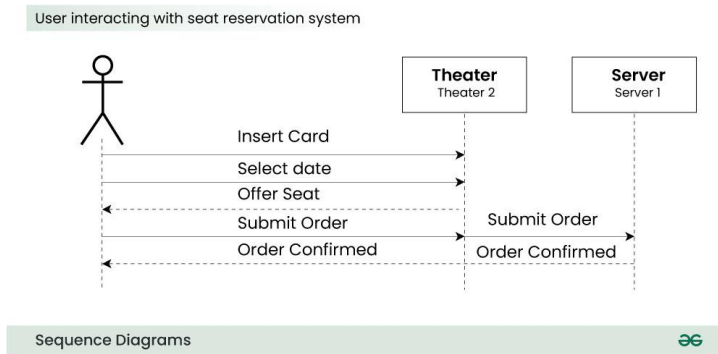
Actors

An **Actor** represents an external entity that interacts with the system. Actors are always outside the system boundary and can be human users or external systems.



Note

Actors represent **roles**, not individual people or objects.

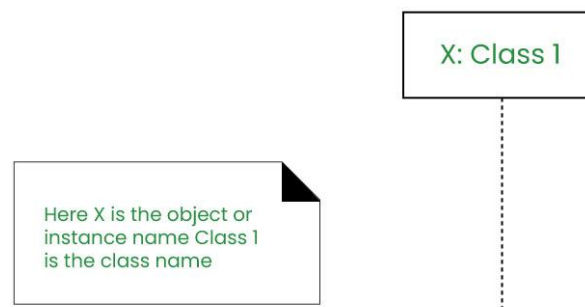


Lifelines

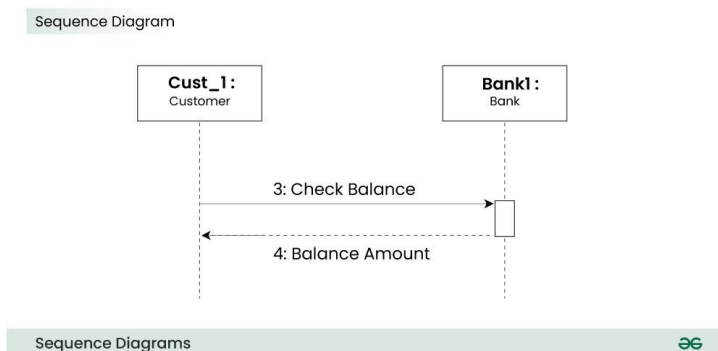
A **Lifeline** represents an object instance participating in the interaction. It consists of:

- Object head (name and type)
- Vertical dashed line (lifetime of the object)

Lifeline

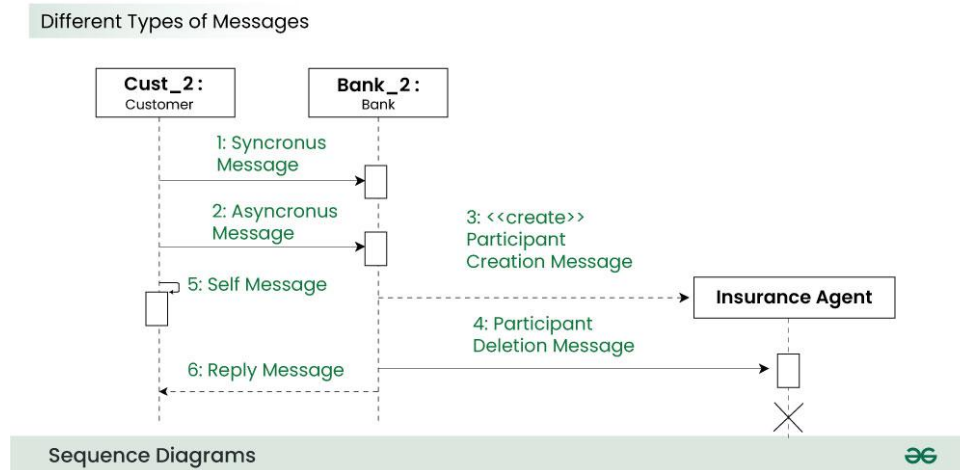


Sequence Diagrams



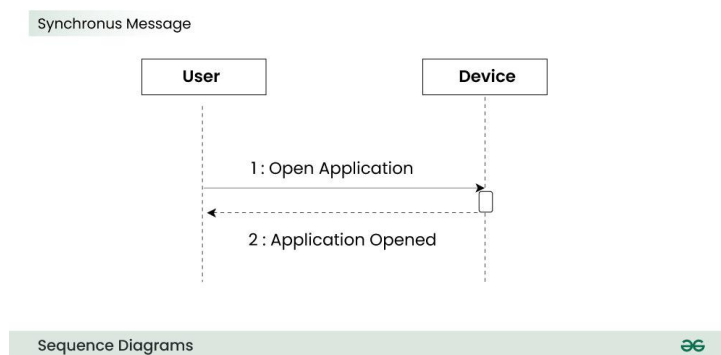
Messages

Messages represent communication between objects and are shown using arrows.

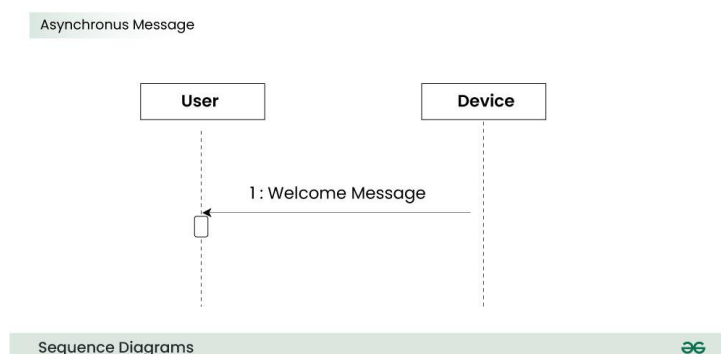


Types of Messages

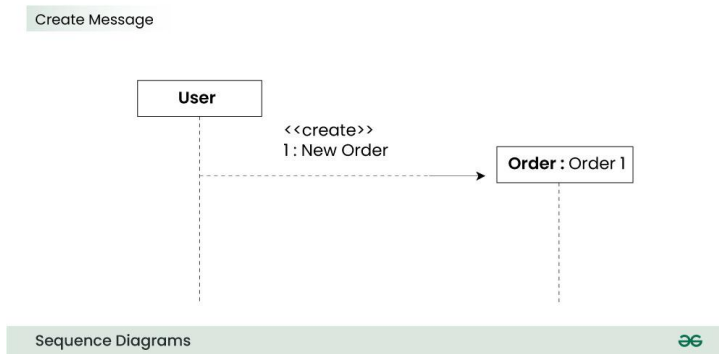
1. Synchronous Message Sender waits until the receiver completes execution.



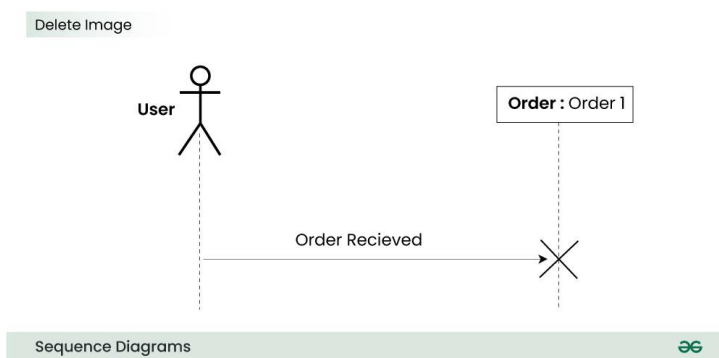
2. Asynchronous Message Sender does not wait for response.



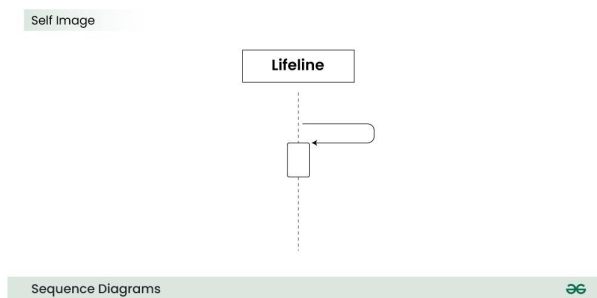
3. Create Message Used to create a new object during interaction.



4. Delete Message Used to destroy an object.



5. Self Message Object sends a message to itself.

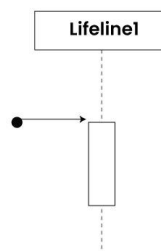


6. Reply Message Response sent back to the caller.



7. Found Message Message received from an unknown source.

Found Message

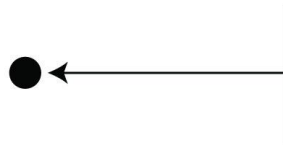


Sequence Diagrams



8. Lost Message Message sent to an unknown destination.

Lost Image



Sequence Diagrams

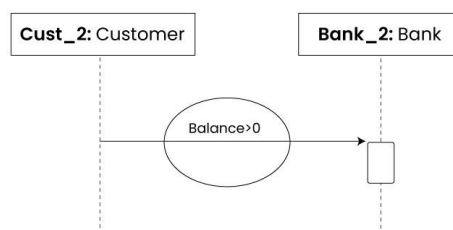


Guards

Guards

Guards are conditional expressions written in square brackets that control the flow of messages in a sequence diagram.

Guards



Sequence Diagrams



Note

Guards are heavily tested in exams — always explain the condition clearly.

Examples

Emotion-Based Music Player

1. User opens the application
2. System accesses webcam
3. Image is captured
4. Face detection and mood analysis performed
5. Mood dictionary requested from database
6. Mood retrieved
7. Mood displayed
8. Music requested
9. Playlist generated and shown

Example sequence diagram

